

Project Document: GenAI Sports Tournament Discovery System

Submitted by: Arjun Gupta

Date: August 3, 2025

1. Introduction & Project Goal

This document outlines the design and implementation of a GenAI-powered system built to fulfill the requirements of the StapuBox GenAI Intern assignment. The primary goal was to engineer a modern, scalable, and intelligent solution for discovering and organizing information about sports tournaments from the web.

The final product is a robust, modular data pipeline that leverages a suite of modern AI tools and development practices to deliver a clean, structured, and up-to-date calendar of sporting events.

2. System Architecture: A Modular, GenAI-First Approach

I designed the system as a "Dynamic Search & Extraction Engine," moving beyond traditional static web scrapers to use AI for the core tasks of discovery and data structuring.

The entire codebase is built as a **modular Python package** (`tournament_system`), promoting separation of concerns and maintainability. The project dependencies and virtual environment are managed using `uv`, the high-performance Python package installer and resolver, ensuring a modern and reproducible development environment.

The key components of the pipeline are:

- **Intelligent Query Generation:** The pipeline begins by generating a diverse set of search queries. This process can be optionally enhanced by an LLM (GPT-3.5) to create more creative and varied search terms.
- **Dynamic Source Discovery (Serper API):** The system uses the Serper API to perform real-time Google searches, ensuring it always works with the most current and relevant web pages.
- **AI-Powered Structured Extraction (FireCrawl API):** This is the core

of the system's intelligence. The pipeline uses FireCrawl's `llm-extraction` mode, passing a predefined Pydantic schema (`TournamentSchema`) to the API. This allows the system to intelligently identify and extract the required data fields directly into a clean JSON format, making the process highly efficient and resilient to website layout changes.

3. Strategic Design & Implementation Choices

a) Top-Down Data Strategy

For this initial build, I deliberately adopted a **top-down data collection strategy**, focusing first on high-quality data from **International tournaments**.

- **Rationale:** This strategic choice was made to build the most robust and accurate foundation possible. International events typically have the most reliable and consistently structured data, published on official governing body websites (e.g., ICC, FIFA, BWF). By proving the pipeline's effectiveness on this high-quality data first, I have established a core system that is proven to work under ideal conditions.
- **Scalability and Future Refinement:** The system is fully capable of targeting national, state, and local tournaments as the next logical phase of development. This expansion is not a matter of difficulty, but one of **time and focused refinement**. Given the time constraints of this assignment, I prioritized building and perfecting the core architecture. Expanding to lower-level tournaments would require dedicated time to:
 - Develop more granular search strategies to handle the noise from local sources.
 - Iteratively refine the extraction prompts and schemas to adapt to less structured web pages.
 - Implement enhanced data validation to ensure quality across varied sources.
- This foundational work on the international tier ensures the system is now perfectly poised for this subsequent expansion.

b) Live Search for GenAI Demonstration

The provided UI and API are configured to perform a **live search**, triggering the entire AI pipeline in real-time.

- **Rationale:** This was a conscious choice to best showcase the system's end-to-end GenAI capabilities. It allows an evaluator to witness the dynamic process of search, discovery, and extraction as it happens. While a production environment would likely run this pipeline as a background job and cache results in a database for performance, the live method provides a transparent demonstration of the core AI engine at work.

4. How to Run the Project

The project is designed for a straightforward setup. For complete, detailed instructions, please refer to the `README.md` file included in the project folder.

1. **Environment Setup:** Create a `.env` file from the `.env.example` template and populate it with the necessary API keys.
2. **Install Dependencies:** Using `uv`, run the command: `uv pip install -r requirements.txt`.
3. **Run the Data Pipeline:** To execute the entire data collection and processing workflow, run the main script: `python run.py`.
4. **Start the Development Servers:** To run both the backend and frontend servers concurrently for development, use the command: `uv run python dev.py`.
 - The **Backend API** will be accessible at `http://localhost:8000`.
 - The **Frontend UI** will be accessible at `http://localhost:3000`.

5. Key Deliverables

- **Modular Codebase:** A clean, well-structured Python application managed with `uv`.
- **Automated Data Pipeline:** The `run.py` script automates the entire data collection process.
- **Structured Data Output:** Final data is saved in both `JSON` and `CSV`

formats in the `/final_output` directory.

- **Bonus API:** A Flask-based REST API that serves the collected tournament data.
- **Bonus UI Mockup:** A functional frontend that fetches and displays the tournament data from the API.